P1748R0
Fill in [delay.cpp] TODO in P1389

Published Proposal, 2019-06-12

This version:
https://yehezkelshb.github.io/cpp_proposals/sg20/P1748-fill-delay-cpp-section-in-P1389.html

Author:
Yehezkel Bernat

Audience:
SG20

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1748 adds wording to replace the TODO of [delay.cpp] section in P1389

Table of Contents

§ 1. Proposed Wording

Under 2.3.5 C Preprocessor [**delay.cpp**]:

~~Excludes `#include`, which is necessary until modules are in C++.~~

~~TODO~~

Most of the traditional usages of the C Preprocessor have better and safer C++ replacements. For

example:

```
// compile-time constant
#define BUFFER_SIZE 256
// better as:
auto constexpr buffer_size = 256;

// named constants
#define RED   0xFF0000
#define GREEN 0x00FF00
#define BLUE  0x0000FF
// better as:
enum class Color { red = 0xFF0000, green = 0x00FF00, blue = 0x0000FF };

// inline function
#define SUCCEEDED(res) (res == 0)
// better as:
inline constexpr bool succeeded(int const res) { return res == 0; }

// generic function
#define MIN(a,b) ((a) < (b) ? (a) : (b))
// better as:
template <typename T>
T const& min(T const& a, T const& b) {
    return a < b ? a : b;
}
// (in actual code, use std::min() instead of reimplementing it)
```

All these macros have many possible pitfalls (see gcc docs), they hard to get right and they don't obey scope and type rules. The C++ replacements are easier to get right and fit better into the general picture.

The only preprocessor usages that are necessary right at the beginning are:

- `#include` for textual inclusion of header files (at least until modules become the main tool for consuming external code)
- `#ifndef` for creating "include guards" to prevent multiple inclusion

§ 2. Acknowledgements